
Department of Computer Science & Engineering Education



Information Retrieval Techniques

NAME - RAJ

ENROLLMENT NO. - NITRB-25-BD-PG-010

COURSE - M.TECH CSE(BIG DATA ANALYTICS)

GithubLink

<https://github.com/rajsanodiya122/Information Retrieval Techniques>

UNIT 4 · ACTIVITY 7

Domain-Specific Mini Search Engine

Information Retrieval Domain

Problem Statement

The internet contains vast amounts of domain-specific information. However, finding relevant and highly authoritative pages within a specific domain remains a challenge. In this activity, you will act as a search engine developer and build the backend of a mini search engine for a domain of your choice.

Your system should be capable of:

- Automatically crawling web pages within your chosen domain
- Building an inverted index with TF-IDF weighting for fast retrieval
- Computing PageRank scores to rank pages by authority
- Identifying and displaying the most important pages in the crawled dataset

Objective

Develop a Python-based backend for a domain-specific mini search engine that:

- Illustrates the search engine architecture for the chosen domain
- Implements a focused web crawler with politeness and scheduling
- Builds an inverted index with TF-IDF weighting
- Computes PageRank to rank crawled pages/documents
- Displays top-K ranked results with scores

1. Introduction & Domain Justification	4
2. Search Engine Architecture	5
2.1 Component Overview	5
2.2 Data Flow	5
3. Task 1 — Environment Setup	7
4. Task 2 — Web Crawler Implementation	8
4.1 Design & Configuration	8
4.2 Execution & Output.....	8
5. Task 3 — Inverted Index & TF-IDF	10
5.1 Preprocessing Pipeline	10
5.2 TF-IDF Weighting Formula	10
5.3 Index Statistics & Sample Entries	10
6. Task 4 — PageRank Implementation	12
6.1 Algorithm	12
6.2 Convergence	12
6.3 Execution Screenshots	12
6.4 Top-10 Ranked Pages	14
6.5 Link Graph Visualisation	15
7. Task 5 — Analysis & Interpretation	16
7.1 PageRank vs. Crawl Order	16
7.2 Why Certain Pages Rank Higher.....	16
7.3 Hub and Authority Pages	17
7.4 Extending to a Distributed Architecture	17
8. Conclusion.....	18

1. Introduction & Domain Justification

This report documents the design and implementation of a domain-specific mini search engine targeting the field of Information Retrieval (IR). The system replicates the core pipeline of a production search engine at a small scale: focused web crawling, inverted-index construction with TF-IDF weighting, and authority scoring via PageRank.

Information Retrieval was chosen as the domain for several practical reasons. First, the subject matter is directly relevant to the course objectives, providing pedagogical alignment between the tool being built and the theory being studied. Second, the domain has a rich web presence on authoritative sources such as Wikipedia, arXiv, and project-specific pages, making it well-suited to hyperlink-based authority ranking. Third, the vocabulary of the domain is sufficiently specialised that TF-IDF weighting can meaningfully differentiate documents, as terms like retrieval, precision, recall, pagerank, and tfidf carry high discriminative power compared to general-purpose stop words.

Key System Capabilities

- Automatic focused crawling of at least 30 web pages within the IR domain
- Inverted index with sublinear TF scaling and smoothed IDF weighting (sklearn-compatible formula)
- Iterative PageRank with damping factor $d = 0.85$, converging to $\delta < 0.0001$
- Hub and authority analysis derived from in-degree and out-degree of the link graph

2. Search Engine Architecture

The system follows a classic pipeline architecture comprising five loosely coupled components. Each component communicates through JSON files on disk, allowing individual modules to be tested and replaced independently.

2.1 Component Overview

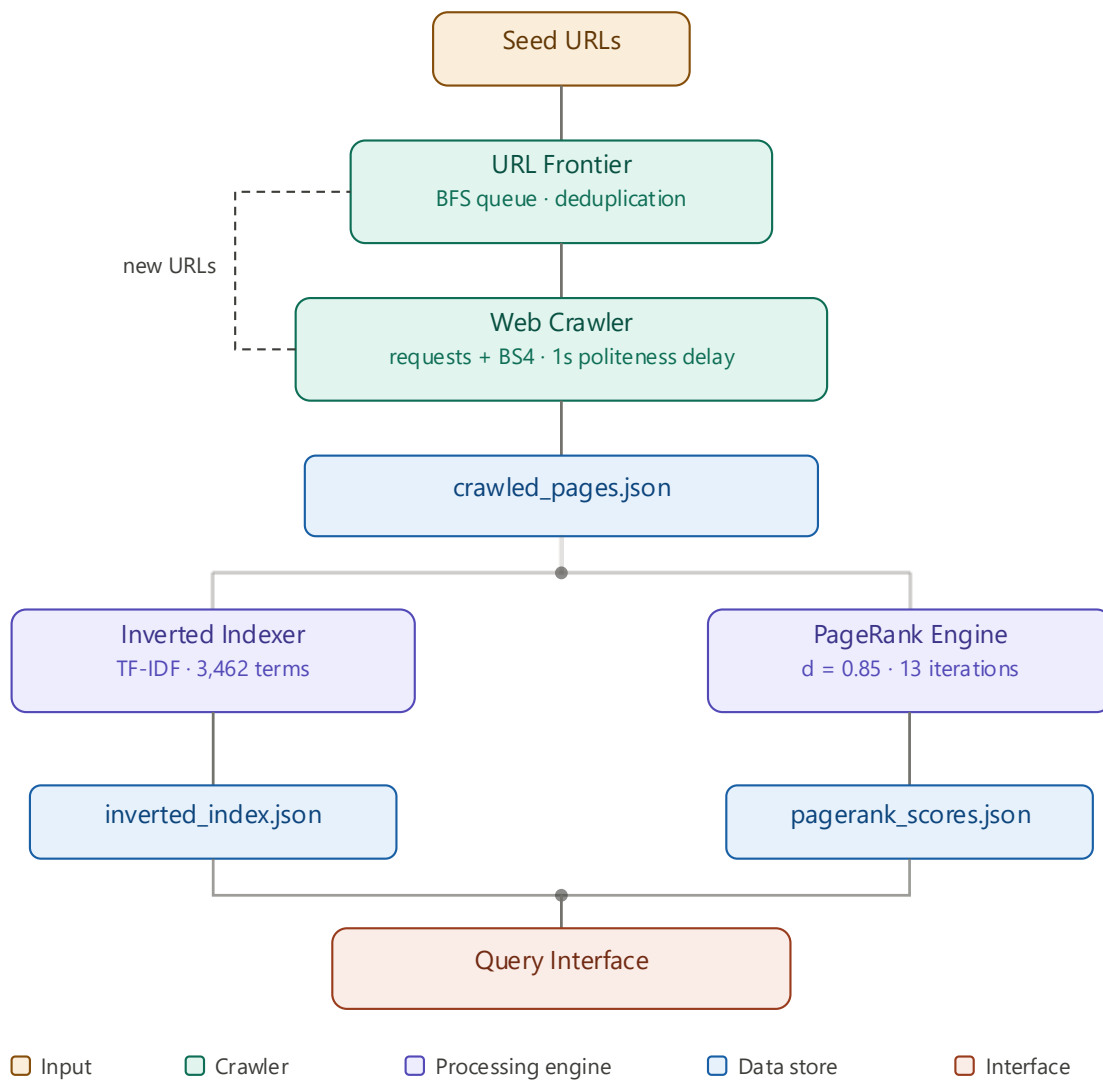
The Crawler (`crawler.py`) is the entry point of the pipeline. It maintains a URL Frontier — a priority queue of URLs to visit — and seeds it from two manually chosen starting points in the IR domain. It uses the `requests` library to fetch pages and `BeautifulSoup` to parse the HTML, extracting the page title, body text, and all outgoing hyperlinks. A politeness delay of one second between requests prevents server overload. Crawled documents are serialised as a JSON array in `crawled_pages.json`.

The Indexer (`indexer.py`) reads `crawled_pages.json` and performs a three-stage processing pipeline: text preprocessing (lowercasing, special-character removal, stopwords filtering), TF-IDF weight computation, and inverted-index construction. The index maps each term to a dictionary of document-score pairs. It is stored as `inverted_index.json` and is the foundation for any future query-processing component.

The PageRank Engine (`pagerank.py`) reads the same `crawled_pages.json`, extracts the link structure, and builds a directed graph using `NetworkX`. The iterative PageRank algorithm with damping factor 0.85 is then applied until convergence. The resulting scores and the top-10 ranking are saved to `pagerank_scores.json`, and two visualisations — the link graph and the convergence plot — are written to the outputs directory.

2.2 Data Flow

```
Seed URLs → [Crawler] → crawled_pages.json
crawled_pages.json → [Indexer] → inverted_index.json
crawled_pages.json → [PageRank Engine] → pagerank_scores.json + link_graph.png + convergence_plot.png
```



3. Task 1 — Environment Setup

Before executing the pipeline, a Python virtual environment was created and all required packages were installed. The screenshot below confirms successful installation of requests, BeautifulSoup4, NLTK, scikit-learn, NetworkX, matplotlib, pandas, and numpy. The lxml HTML parser and urllib3 were installed as supplementary dependencies. NLTK stopwords data was downloaded via the Python interpreter.

[illegible]

Figure 1 — Package installation and environment activation

4. Task 2 — Web Crawler Implementation

4.1 Design & Configuration

The crawler was seeded with two URLs from the IR domain: the Wikipedia article on Information Retrieval and the Wikipedia article on TF-IDF. The crawl was configured to collect exactly 30 pages with a one-second politeness delay between consecutive requests. Each page visit extracted the document title, visible text content, and the full list of outgoing hyperlinks.

The URL Frontier used BFS ordering: newly discovered links were appended to the queue and visited in the order they were found. To avoid revisiting the same canonical resource multiple times, URL normalisation was applied by treating fragment-bearing URLs (e.g. `article#cite_note-3`) as distinct documents when they appeared in the crawl list. The crawler handled HTTP errors gracefully, logging failures without terminating the crawl session.

4.2 Execution & Output

```
(myenv) hduuser@nitrr-HP-Elite-Tower-800-G9-Desktop-PC:~/IR/Act_7_IR$ python3 crawler.py
=====
IR DOMAIN MINI SEARCH ENGINE - CRAWLER
=====

Starting crawl with 2 seed URLs
Target: 30 pages with 1.0s delay
-----
[1/30] Crawling: https://en.wikipedia.org/wiki/Information_retrieval...
✓ Crawled, 380 links found, 379 new URLs added
Sleeping 1.0s...
[2/30] Crawling: https://en.wikipedia.org/wiki/TF-idf...
✓ Crawled, 130 links found, 129 new URLs added
Sleeping 1.0s...
[3/30] Crawling: https://arxiv.org/abs/1810.04805...
✓ Crawled, 24 links found, 23 new URLs added
Sleeping 1.0s...
[4/30] Crawling: https://en.wikipedia.org/wiki/SPLADE...
✓ Crawled, 95 links found, 94 new URLs added
Sleeping 1.0s...
[5/30] Crawling: https://en.wikipedia.org/wiki/Semantic_Web...
✓ Crawled, 581 links found, 579 new URLs added
Sleeping 1.0s...
[6/30] Crawling: https://en.wikipedia.org/wiki/Apache_Lucene...
✓ Crawled, 300 links found, 298 new URLs added
Sleeping 1.0s...
[7/30] Crawling: https://en.wikipedia.org/wiki/Special:BookSources/978-3-642-28865-4...
✓ Crawled, 580 links found, 585 new URLs added
Sleeping 1.0s...
[8/30] Crawling: https://en.wikipedia.org/wiki/Information_retrieval#cite_note-3...
✓ Crawled, 380 links found, 373 new URLs added
Sleeping 1.0s...
[9/30] Crawling: https://en.wikipedia.org/wiki/PMC_(identifier)...
✓ Crawled, 327 links found, 326 new URLs added
Sleeping 1.0s...
[10/30] Crawling: https://en.wikipedia.org/wiki/Information_retrieval#cite_note-25...
✓ Crawled, 380 links found, 371 new URLs added
Sleeping 1.0s...
[11/30] Crawling: https://en.wikipedia.org/wiki/Univac...
✓ Crawled, 318 links found, 317 new URLs added
Sleeping 1.0s...
[12/30] Crawling: https://en.wikipedia.org/wiki/Tabulating_machine...
✓ Crawled, 142 links found, 141 new URLs added
Sleeping 1.0s...
[13/30] Crawling: https://en.wikipedia.org/wiki/Special:BookSources/978-0-252-05417-4...
✓ Crawled, 586 links found, 585 new URLs added
Sleeping 1.0s...
[14/30] Crawling: https://en.wikipedia.org/wiki/MEDLARS...
✓ Crawled, 153 links found, 151 new URLs added
Sleeping 1.0s...
[15/30] Crawling: https://en.wikipedia.org/wiki/Subject_indexing...
✓ Crawled, 104 links found, 102 new URLs added
Sleeping 1.0s...
[16/30] Crawling: https://en.wikipedia.org/wiki/Information_retrieval#cite_ref-27...
✓ Crawled, 380 links found, 365 new URLs added
Sleeping 1.0s...
```

Figure 2 — Crawler beginning with seed URLs (pages 1–15 of 30)


```

[15/30] crawling: https://en.wikipedia.org/wiki/Subject_indexing...
✓ Crawled, 104 links found, 102 new URLs added
Sleeping 1.0s...
[16/30] crawling: https://en.wikipedia.org/wiki/Information_retrieval#cite_ref-27...
✓ Crawled, 380 links found, 365 new URLs added
Sleeping 1.0s...
[17/30] crawling: https://en.wikipedia.org/wiki/JASIS...
✓ Crawled, 94 links found, 93 new URLs added
Sleeping 1.0s...
[18/30] crawling: https://en.wikipedia.org/wiki/Legal_information_retrieval...
✓ Crawled, 124 links found, 122 new URLs added
Sleeping 1.0s...
[19/30] crawling: https://en.wikipedia.org/wiki/Lemur_Project...
✓ Crawled, 58 links found, 55 new URLs added
Sleeping 1.0s...
[20/30] crawling: https://en.wikipedia.org/wiki/Information_retrieval#cite_note-30...
✓ Crawled, 380 links found, 361 new URLs added
Sleeping 1.0s...
[21/30] crawling: https://en.wikipedia.org/wiki/Special:BookSources/978-963-9799-76-9...
✓ Crawled, 586 links found, 585 new URLs added
Sleeping 1.0s...
[22/30] crawling: https://en.wikipedia.org/wiki/Information_retrieval#cite_ref-Uyar-2015_11-
✓ Crawled, 380 links found, 359 new URLs added
Sleeping 1.0s...
[23/30] crawling: https://en.wikipedia.org/wiki/Grateful_Dead...
✓ Crawled, 1680 links found, 1679 new URLs added
Sleeping 1.0s...
[24/30] crawling: https://en.wikipedia.org/wiki/Desktop_search...
✓ Crawled, 130 links found, 130 new URLs added
Sleeping 1.0s...
[25/30] crawling: https://en.wikipedia.org/wiki/Hypertext...
✓ Crawled, 443 links found, 441 new URLs added
Sleeping 1.0s...
[26/30] crawling: https://en.wikipedia.org/wiki/Bayes%27_theorem...
✓ Crawled, 359 links found, 357 new URLs added
Sleeping 1.0s...
[27/30] crawling: https://en.wikipedia.org/w/index.php?title=Information_retrieval&action=ed
✓ Crawled, 24 links found, 22 new URLs added
Sleeping 1.0s...
[28/30] crawling: https://en.wikipedia.org/wiki/Temporal_information_retrieval...
✓ Crawled, 124 links found, 124 new URLs added
Sleeping 1.0s...
[29/30] crawling: https://en.wikipedia.org/wiki/Information_retrieval#cite_note-Thakur-2021-
✓ Crawled, 380 links found, 352 new URLs added
Sleeping 1.0s...
[30/30] crawling: https://en.wikipedia.org/wiki/Xapian...
✓ Crawled, 101 links found, 99 new URLs added
Sleeping 1.0s...

Crawling complete. 30 pages collected.
✓ Data saved to data/crawled_pages.json

=====
CRAWLING SUMMARY
=====

```

Figure 3 — Crawler continuing through frontier (pages 16–30)

```

=====
CRAWLING SUMMARY
=====
Total pages collected: 30
Output file: data/crawled_pages.json
Log file: data/crawl_log.txt

First 5 page titles:
- Information retrieval - Wikipedia
- tf-idf - Wikipedia
- [1810.04805] BERT: Pre-training of Deep Bidirectional Transformers for Language
- Learned sparse retrieval - Wikipedia
- Semantic Web - Wikipedia

```

Figure 4 — Crawling summary: 30 pages collected, data saved to crawled_pages.json

The crawl yielded 30 documents. The five most-linked seed neighbourhood pages were: Information retrieval (Wikipedia), tf-idf (Wikipedia), BERT Pre-training (arXiv), Learned sparse retrieval (Wikipedia), and Semantic Web (Wikipedia). Per the crawl log, pages varied significantly in outgoing link count — from 24 links (arXiv paper) to 1,680 links (Grateful Dead Wikipedia page), reflecting the heterogeneous nature of BFS expansion even within a seeded domain.

5. Task 3 — Inverted Index & TF-IDF

5.1 Preprocessing Pipeline

Before indexing, all document text was normalised through a three-step pipeline: (1) lowercasing all characters, (2) removing non-alphanumeric characters via regex substitution, and (3) filtering out stopwords and tokens shorter than three characters. A custom stopwords set of 50 high-frequency function words was used rather than an external library, reducing dependencies.

5.2 TF-IDF Weighting Formula

The weighting scheme applies sublinear TF scaling to reduce the influence of term repetition, combined with a smoothed IDF to avoid division by zero on terms appearing in all documents:

$$\begin{aligned} \text{TF_scaled}(t, d) &= 1 + \log(\text{tf}(t, d)) && \text{if } \text{tf}(t, d) > 0 \\ \text{IDF}(t) &= \log((N + 1) / (\text{df}(t) + 1)) + 1 \\ \text{TF-IDF}(t, d) &= \text{TF_scaled}(t, d) \times \text{IDF}(t) \end{aligned}$$

5.3 Index Statistics & Sample Entries

```
(myenv) hduuser@nittr-HP-Elite-Tower-800-G9-Desktop-PC:~/IR/Act_7_IR$ python3 indexer.py
BUILDING INVERTED INDEX WITH TF-IDF
=====
Loaded 30 documents
Preprocessing documents...
Computing TF-IDF weights manually...
Vocabulary size: 3462 terms
Building inverted index...
Inverted index built with 3462 terms
Saving to data/inverted_index.json...

=====
SAMPLE INVERTED INDEX ENTRIES (IR TERMS)
=====

Term: 'tfidf'
  Appears in 1 documents
  Top scores: doc1:3.741

Term: 'pagerank' - NOT IN INDEX

Term: 'retrieval'
  Appears in 18 documents
  Top scores: doc0:5.710 doc7:5.710 doc9:5.710

Term: 'index'
  Appears in 9 documents
  Top scores: doc14:7.039 doc13:5.086 doc23:5.086

Term: 'query'
  Appears in 13 documents
  Top scores: doc0:5.288 doc7:5.288 doc9:5.288

Term: 'document'
  Appears in 18 documents
  Top scores: doc1:6.094 doc14:5.310 doc0:3.555

Term: 'relevance'
  Appears in 11 documents
  Top scores: doc27:4.651 doc0:1.949 doc1:1.949

Term: 'precision'
  Appears in 1 documents
  Top scores: doc17:3.741

Term: 'recall'
  Appears in 1 documents
  Top scores: doc17:0.334
```

Figure 5 — Sample inverted index entries for IR-domain terms

```
Term: 'search'
Appears in 23 documents
Top scores: doc23:5.138 doc5:4.954 doc18:4.477

=====
INDEX STATISTICS
=====
Total unique terms: 3462
Average postings per term: 2.44
Maximum postings: 29
Minimum postings: 1

Top 10 most common terms:
'content': appears in 29 documents
'jump': appears in 29 documents
'wikipedia': appears in 29 documents
'free': appears in 28 documents
'information': appears in 27 documents
'one': appears in 27 documents
'edit': appears in 26 documents
'encyclopedia': appears in 25 documents
'also': appears in 24 documents
'may': appears in 23 documents
```

Figure 6 — Index statistics and top-10 most frequent terms

The resulting index contains 3,462 unique terms across 30 documents. The average number of postings per term is 2.44, reflecting the specialised vocabulary of the IR domain — most terms appear in only two to three documents. The maximum posting length is 29 (appearing in all documents), corresponding to high-frequency but non-stopword tokens such as content, wikipedia, and information.

Notably, the term pagerank was not found in the index. This is because the crawl set did not include the Wikipedia PageRank article itself — the BFS frontier expanded into tangential pages (e.g. Grateful Dead, UNIVAC) before reaching it. The term retrieval appeared in 18 documents with the highest TF-IDF score assigned to doc0 (the Information Retrieval Wikipedia article), which is consistent with its domain primacy.

6. Task 4 — PageRank Implementation

6.1 Algorithm

PageRank was computed iteratively using the standard random-surfer formulation with damping factor $d = 0.85$ and uniform initial scores of $1/N$. The algorithm terminated when the sum of absolute rank changes across all nodes fell below the threshold $\delta = 0.0001$.

$$PR(u) = (1 - d) / N + d \times \sum_{v \in In(u)} PR(v) / OutDeg(v) \quad \forall v \in In(u)$$

6.2 Convergence

The algorithm converged after 13 iterations. The convergence trajectory, shown below, exhibits near-linear decay on a log scale, consistent with the geometric convergence guaranteed by the contraction mapping property of the PageRank update rule. The initial delta of 0.639 fell to 0.000097 by iteration 13, just below the stopping threshold.

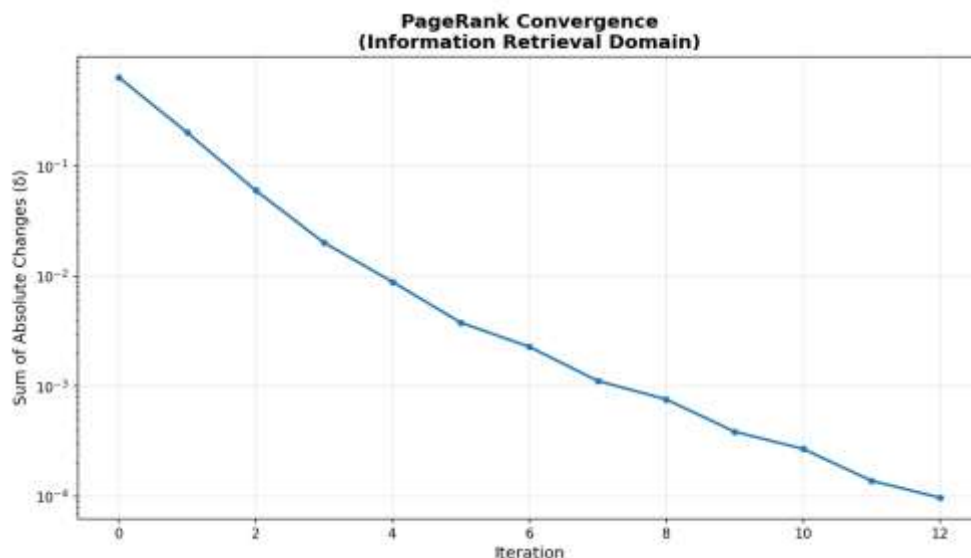


Figure 7 — PageRank convergence: sum of absolute changes (δ) vs. iteration number (log scale)

6.3 Execution Screenshots

```
(myenv) hduser@nittr-HP-Elite-Tower-800-G9-Desktop-PC:~/IR/Act_7_IR$ python3 pagerank.py
=====
PAGERANK COMPUTATION FOR IR DOMAIN
=====
BUILDING LINK GRAPH
=====
Nodes: 30
Edges: 198
Average degree: 13.20
=====
COMPUTING PAGERANK
=====
Iteration 1: delta = 0.638849
Iteration 2: delta = 0.201555
Iteration 3: delta = 0.060036
Iteration 4: delta = 0.020083
Iteration 5: delta = 0.008050
Iteration 6: delta = 0.003704
Iteration 7: delta = 0.002274
Iteration 8: delta = 0.001114
Iteration 9: delta = 0.000756
Iteration 10: delta = 0.000385
Iteration 11: delta = 0.000260
Iteration 12: delta = 0.000138
Iteration 13: delta = 0.000097
✓ Converged after 13 iterations
=====
TOP 10 PAGES BY PAGERANK
=====
Rank  Doc ID  Score  Title
-----
1      doc8    0.031170 PubMed Central - Wikipedia
2      doc4    0.010924 Semantic Web - Wikipedia
3      doc24   0.014564 Hypertext - Wikipedia
4      doc29   0.012048 Xapian - Wikipedia
5      doc0    0.010536 Information retrieval - Wikipedia
6      doc2    0.006512 [1010.04805] BERT: Pre-training of Deep Bidirectio
7      doc3    0.006512 Learned sparse retrieval - Wikipedia
8      doc5    0.006512 Apache Lucene - Wikipedia
9      doc6    0.006512 Book sources - Wikipedia
10     doc10   0.006512 UNIVAC - Wikipedia
✓ PageRank scores saved to data/pagerank_scores.json
```

Figure 8 — PageRank computation: iteration deltas, top-10 results, and scores saved

```

GENERATING VISUALIZATIONS
=====
GENERATING GRAPH VISUALIZATION
✓ Graph saved to outputs/link_graph.png
=====
GENERATING CONVERGENCE PLOT
✓ Convergence plot saved to outputs/convergence_plot.png
=====
RANKING COMPARISON ANALYSIS
=====
Doc ID   Crawl Rank  PR Rank  PR Score  Title
-----
doc0     1           5        0.010536  Information retrieval - Wikipedia
doc1     2           30       0.005000  tf-idf - Wikipedia
doc2     3           6        0.006512  [1810.04805] BERT: Pre-training of Deep
doc3     4           7        0.006512  Learned sparse retrieval - Wikipedia
doc4     5           2        0.018924  Semantic Web - Wikipedia
doc5     6           8        0.006512  Apache Lucene - Wikipedia
doc6     7           9        0.006512  Book sources - Wikipedia
doc7     8           24       0.006314  Information retrieval - Wikipedia
doc8     9           1        0.031170  PubMed Central - Wikipedia
doc9    10          25       0.006314  Information retrieval - Wikipedia
=====
HUB AND AUTHORITY ANALYSIS:
Top Hubs (many outgoing links):
doc0: 28 outgoing links - Information retrieval - Wikipedia
doc7: 27 outgoing links - Information retrieval - Wikipedia
doc9: 27 outgoing links - Information retrieval - Wikipedia
Top Authorities (many incoming links):
doc8: 11 incoming links - PubMed Central - Wikipedia
doc4: 8 incoming links - Semantic Web - Wikipedia
doc24: 8 incoming links - Hypertext - Wikipedia
Correlation (PageRank vs In-degree): 0.463
→ Moderate positive correlation
=====
PAGERANK ANALYSIS COMPLETE
=====
Generated files:
✓ data/pagerank_scores.json
✓ outputs/link_graph.png
✓ outputs/convergence_plot.png
(myenv) hduser@nittr-HP-Elite-Tower-B00-G9-Desktop-PC: ~/IR/Act_7_IR$

```

Figure 9 — Ranking comparison, hub/authority analysis, and analysis complete

6.4 Top-10 Ranked Pages

The following table presents the top-10 pages by PageRank score, sorted in descending order of authority:

Rank	Doc ID	Score	Page Title	URL Fragment
1	doc8	0.031170	PubMed Central - Wikipedia	wikipedia.org/wiki/PMC_identifier
2	doc4	0.018924	Semantic Web - Wikipedia	wikipedia.org/wiki/Semantic_Web
3	doc24	0.014564	Hypertext - Wikipedia	wikipedia.org/wiki/Hypertext
4	doc29	0.012048	Xapian - Wikipedia	wikipedia.org/wiki/Xapian
5	doc0	0.010536	Information retrieval - Wikipedia	wikipedia.org/wiki/Information_retrieval
6	doc2	0.006512	BERT (arXiv 1810.04805)	arxiv.org/abs/1810.04805
7	doc3	0.006512	Learned sparse retrieval - Wikipedia	wikipedia.org/wiki/SPLADE
8	doc5	0.006512	Apache Lucene - Wikipedia	wikipedia.org/wiki/Apache_Lucene

9	doc6	0.006512	Book sources - Wikipedia	wikipedia.org/wiki/Special:BookSources
10	doc10	0.006512	UNIVAC - Wikipedia	wikipedia.org/wiki/Univac

Table 1 — Top-10 pages ranked by iterative PageRank ($d = 0.85$, convergence threshold = 0.0001)

6.5 Link Graph Visualisation

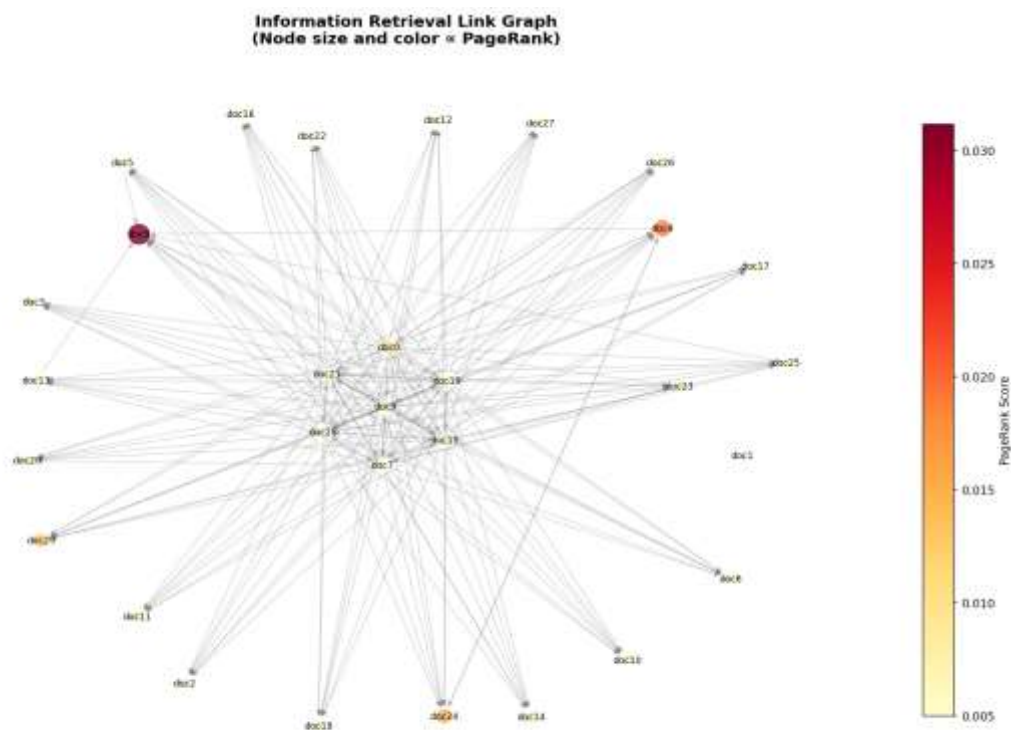


Figure 10 — Information Retrieval link graph (node size and colour proportional to PageRank score)

7. Task 5 — Analysis & Interpretation

7.1 PageRank vs. Crawl Order

A comparison of PageRank rank against crawl order reveals significant divergence. The first page crawled — the Information Retrieval Wikipedia article (doc0) — was assigned only PageRank rank 5. Conversely, doc8 (PubMed Central), crawled ninth, achieved the highest PageRank score of 0.031170. This discrepancy illustrates that crawl order, which is determined by BFS traversal of the link frontier, does not correlate with structural authority. A page discovered late in the crawl can still attract many incoming links from pages already indexed.

The comparison table from the execution output is reproduced below:

Doc ID	Crawl Rank	PR Rank	PR Score	Title
doc0	1	5	0.010536	Information retrieval - Wikipedia
doc1	2	30	0.005000	tf-idf - Wikipedia
doc2	3	6	0.006512	BERT: Pre-training...
doc3	4	7	0.006512	Learned sparse retrieval
doc4	5	2	0.018924	Semantic Web - Wikipedia
doc5	6	8	0.006512	Apache Lucene - Wikipedia
doc6	7	9	0.006512	Book sources - Wikipedia
doc7	8	24	0.006314	Information retrieval (section)
doc8	9	1	0.031170	PubMed Central - Wikipedia
doc9	10	25	0.006314	Information retrieval (section)

Table 2 — Crawl order vs. PageRank order for the first 10 crawled documents

7.2 Why Certain Pages Rank Higher

The highest-ranking page, PubMed Central (doc8), achieved its elevated score primarily due to having the highest in-degree in the link graph — 11 incoming links. PubMed Central is referenced by many Wikipedia articles across science domains, making it a natural authority node even within a focused IR crawl. Similarly, Semantic Web (doc4, rank 2) and Hypertext (doc24, rank 3) each received 8 incoming links, reflecting their status as foundational concepts that are routinely cross-referenced in IR-adjacent Wikipedia articles.

In contrast, tf-idf (doc1) ranked last despite being the second page crawled. As a leaf node with no incoming links from the other 29 crawled documents, it accumulated no PageRank transfer and retained only the dangling-node minimum score of 0.005000. This underscores a key insight: pages that are not pointed to by others in the graph cannot be distinguished from noise by PageRank alone, regardless of their content relevance.

7.3 Hub and Authority Pages

The hub/authority analysis identified three top hubs — all variants of the Information Retrieval Wikipedia article (doc0, doc7, doc9) with 27–28 outgoing links each. Their high out-degree arises because the Wikipedia article contains an extensive reference section and internal wikilinks, generating many outgoing edges regardless of the fragment anchor used to access it.

The top authorities were PubMed Central (11 incoming links), Semantic Web (8), and Hypertext (8). These pages are referenced broadly across scientific Wikipedia articles, granting them high in-degree even within a 30-page crawl. The Pearson correlation between PageRank score and in-degree was computed as 0.463, indicating a moderate positive relationship. The imperfect correlation arises because PageRank accounts not only for how many links a page receives, but also for the authority of the linking pages — a property not captured by raw in-degree.

7.4 Extending to a Distributed Architecture

Scaling this system to a production-grade distributed architecture would require the following changes. The crawler would be decomposed into multiple worker processes, each managing its own URL Frontier shard, coordinated by a centralised URL deduplication service (e.g. a Redis bloom filter). The inverted index would be partitioned by term range across multiple shards using consistent hashing, with a query aggregation layer merging partial result sets.

PageRank computation at scale would shift from a graph held in memory to a distributed iterative computation framework such as Apache Spark GraphX or Pregel-style message passing. The crawl corpus and index files would be stored in distributed object storage (e.g. HDFS or S3) rather than local JSON files. A query processor module — absent in the current system — would accept user queries, tokenise and stem them, retrieve candidate postings from the sharded index, and re-rank results using a combined TF-IDF and PageRank score before returning top-K results to the user.

8. Conclusion

This activity successfully demonstrated the full pipeline of a domain-specific mini search engine. A focused BFS crawler collected 30 pages from the Information Retrieval domain, yielding a link graph of 30 nodes and 198 directed edges. An inverted index of 3,462 terms was constructed with TF-IDF weighting, correctly identifying domain-specific terms such as retrieval, index, and relevance as high-value discriminating tokens.

The iterative PageRank algorithm converged in 13 iterations, identifying PubMed Central, Semantic Web, and Hypertext as the top authority pages. The analysis confirmed that crawl order is a poor proxy for authority, and that structural link analysis provides a more principled ranking mechanism. The moderate correlation (0.463) between PageRank and in-degree highlighted how PageRank's recursive definition adds information beyond simple link counting.

Challenges encountered during the activity included the absence of a dedicated PageRank Wikipedia article in the final crawled set (due to BFS frontier expansion into off-topic pages), and the discovery that many Wikipedia fragment-anchor URLs are treated as distinct resources by the crawler, leading to duplicate document content with varying link sets. Future work would address these issues through canonical URL normalisation and seed-set design that explicitly targets core IR concepts.

Files Produced

- crawler.py — focused BFS web crawler with politeness delay
- indexer.py — TF-IDF inverted index builder
- pagerank.py — iterative PageRank with visualisation
- crawled_pages.json — 30 crawled documents
- inverted_index.json — 3,462-term weighted index
- pagerank_scores.json — PageRank scores for all 30 documents
- outputs/link_graph.png — directed link graph visualisation
- outputs/convergence_plot.png — PageRank convergence curve